

Guide d'Installation

Thesium Desk

Version 1.0 — Mars 2026

Thesium.finance

Ce document décrit la procédure complète d'installation et de configuration de l'application Thesium Desk, plateforme de gestion de portefeuille propriétaire avec moteur de risque intégré et agents de recherche IA.

Architecture : Backend FastAPI (Python) + Frontend Vanilla JS + Base SQLite

Auteur	Rédacteur technique
Date de création	Mars 2026
Dernière mise à jour	4 mars 2026
Statut	Version 1.0 — Finale

Table des matières

- | | |
|----|---------------------------------------|
| 1. | Introduction |
| 2. | Prérequis Système |
| 3. | Structure du Projet |
| 4. | Installation Pas à Pas |
| 5. | Configuration |
| 6. | Déploiement en Production |
| 7. | Migration vers PostgreSQL (optionnel) |
| 8. | Vérification Post-Installation |
| 9. | Dépannage |

1. Introduction

1.1 Présentation du projet

Thesium Desk est une plateforme de gestion de portefeuille propriétaire conçue pour les équipes d'investissement. Elle intègre un **moteur de risque** (Value at Risk paramétrique), des **agents de recherche IA** spécialisés, et un **cycle de décision automatisé** (paper broker) permettant de simuler des opérations de marché.

L'application est construite autour d'une architecture légère et modulaire, idéale pour un MVP :

1.2 Architecture technique

- **Backend** : FastAPI (Python) — API RESTful performante et asynchrone
- **Frontend** : Vanilla JavaScript — SPA légère sans dépendance framework
- **Base de données** : SQLite — zéro configuration, 12 tables relationnelles
- **Données de marché** : Yahoo Finance (ingestion automatique)
- **Agents IA** : 4 agents spécialisés (Macro, Fondamental, Technique, Sentiment)

1.3 Portée de ce guide

Ce guide couvre l'installation locale complète, la configuration initiale, et les options de déploiement en production. Il est destiné aux développeurs ayant une connaissance de base de Python et de la ligne de commande.

2. Prérequis Système

Avant de commencer l'installation, vérifiez que votre environnement satisfait les conditions suivantes :

Composant	Version minimum	Notes
Python	3.10+	Recommandé 3.11+
pip	21+	Gestionnaire de paquets Python
Navigateur	Chrome, Firefox, Edge	Dernière version
Système	Linux, macOS, Windows	WSL recommandé sous Windows
RAM	2 Go minimum	4 Go recommandé
Disque	500 Mo	Code, base et données
Réseau	Accès internet	Pour Yahoo Finance (optionnel)

Note : Pour le MVP, aucune clé API externe n'est requise. L'application fonctionne entièrement avec des données synthétiques pré-générées.

3. Structure du Projet

Le projet est organisé dans un répertoire unique contenant l'ensemble des fichiers nécessaires au fonctionnement de l'application :

```
thesium-desk/  
|-- api_server.py # Serveur FastAPI principal (port 8000)  
|-- models.py # Schema SQLite (12 tables)  
|-- agents.py # 4 agents IA de recherche  
|-- risk_engine.py # Moteur de risque (VaR paramétrique)  
|-- execution_engine.py # Paper broker + cycle de décision  
|-- memo_generator.py # Générateur de memos IC  
|-- data_ingestion.py # Ingestion Yahoo Finance  
|-- seed_data.py # Données de démonstration  
|-- requirements.txt # Dépendances Python  
|-- thesium.db # Base SQLite (créée au démarrage)  
|-- index.html # Frontend SPA  
|-- base.css # Styles de base  
|-- style.css # Styles applicatifs  
|-- app.js # Logique frontend (vanilla JS)  
`-- RUNBOOK.md # Documentation opérationnelle
```

Fichiers clés :

- **api_server.py** — Point d'entrée principal. Démarre le serveur FastAPI sur le port 8000 et expose tous les endpoints REST.
- **models.py** — Définit le schéma de la base de données SQLite avec 12 tables relationnelles (instruments, positions, ordres, thèses, etc.).
- **agents.py** — Implémente les 4 agents de recherche IA : Macro, Fondamental, Technique et Sentiment.
- **risk_engine.py** — Moteur de calcul de risque intégré (VaR paramétrique, corrélations, limites).
- **seed_data.py** — Génère les données de démonstration (instruments, prix, positions, thèses).

4. Installation Pas à Pas

Suivez ces étapes dans l'ordre pour installer Thesium Desk sur votre machine locale.

1 Étape 1 : Cloner ou copier le projet

Placez tous les fichiers du projet dans un répertoire dédié. Par exemple :

```
mkdir ~/thesium-desk
cd ~/thesium-desk
# Copier tous les fichiers du projet ici
```

2 Étape 2 : Créer un environnement virtuel (recommandé)

L'utilisation d'un environnement virtuel Python est fortement recommandée pour isoler les dépendances du projet :

```
cd thesium-desk
python -m venv venv
source venv/bin/activate # Linux/macOS
# ou: venv\Scripts\activate # Windows
```

3 Étape 3 : Installer les dépendances

Installez toutes les dépendances Python nécessaires via pip :

```
pip install -r requirements.txt
```

Dépendances principales : fastapi, uvicorn (et leurs sous-dépendances automatiques).

4 Étape 4 : Initialiser la base de données et les données de démo

```
python seed_data.py
```

Cette commande crée le fichier `thesium.db` et le remplit avec :

- **14 instruments** : AAPL, MSFT, GOOGL, AMZN, NVDA, TSLA, META, JPM, BAC, XOM, JNJ, UNH, SPY, QQQ
- **30 jours d'historique** de prix par instrument
- **Portefeuille initial** : ~\$1.16M AUM, \$587K cash, 6 positions
- **Thèses actives** de chaque agent de recherche
- **3-4 mémos IC** historiques
- **Journal d'événements** complet

5 Étape 5 : Démarrer le serveur

```
python api_server.py
```

Le serveur démarre sur `http://0.0.0.0:8000`

Note : Au démarrage, le serveur initialise automatiquement la base de données et exécute le seed si nécessaire.

6 Étape 6 : Vérifier le fonctionnement

Testez l'état de santé de l'API :

```
curl http://localhost:8000/api/health
```

Réponse attendue :

```
{"status": "healthy", "timestamp": "...", "service": "Thesium.finance API"}
```

7 Étape 7 : Accéder au frontend

Deux options pour accéder à l'interface utilisateur :

- **En local** : Ouvrir directement `index.html` dans un navigateur
- **À distance** : Configurer le placeholder `API_BASE` dans `app.js` pour pointer vers l'URL du backend

5. Configuration

5.1 Variables d'environnement (MVP)

Dans sa version MVP, Thesium Desk ne nécessite **aucune clé API**. L'application utilise des données synthétiques générées par `seed_data.py`. SQLite fonctionne en mode zéro-configuration.

5.2 Pour la production (futures intégrations)

Les variables suivantes seront nécessaires pour les intégrations futures :

Variable	Description	Exemple
POLYGON_API_KEY	Données temps réel polygon.io	pk_XXXXX
ALPHA_VANTAGE_KEY	Source alternative de données	XXXXXX
IBKR_ACCOUNT	Compte Interactive Brokers	DU12345
IBKR_HOST	Hôte TWS	127.0.0.1
IBKR_PORT	Port TWS	7497
DATABASE_URL	PostgreSQL (migration)	postgresql://...
LIVE_TRADING	Activer le trading réel	true/false

5.3 Configuration du port

Le port par défaut est **8000**. Pour le modifier, changez la dernière ligne du fichier `api_server.py` :

```
# Dernière ligne de api_server.py
uvicorn.run(app, host="0.0.0.0", port=8000) # Modifier le port ici
```


6. Déploiement en Production

6.1 Avec Uvicorn (simple)

```
uvicorn api_server:app --host 0.0.0.0 --port 8000 --workers 1
```

Note : SQLite ne supporte qu'un seul worker. Pour du multi-worker, migrez vers PostgreSQL (voir section 7).

6.2 Avec systemd (Linux)

Créez un fichier de service systemd pour gérer le démarrage automatique :

```
[Unit]
Description=Thesium Desk API
After=network.target

[Service]
User=thesium
WorkingDirectory=/opt/thesium-desk
ExecStart=/opt/thesium-desk/venv/bin/uvicorn api_server:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

Commandes d'activation :

```
sudo cp thesium-desk.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable thesium-desk
sudo systemctl start thesium-desk
```

6.3 Avec Docker (optionnel)

Exemple de Dockerfile pour conteneuriser l'application :

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python seed_data.py
EXPOSE 8000
CMD ["uvicorn", "api_server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Construction et lancement :

```
docker build -t thesium-desk .
docker run -d -p 8000:8000 --name thesium thesium-desk
```

6.4 Servir le frontend

Plusieurs options pour servir les fichiers statiques du frontend :

- **Option A** : Utiliser un serveur web statique (nginx, Caddy) pointant vers les fichiers HTML/CSS/JS
- **Option B** : Modifier `API_BASE` dans `app.js` pour pointer vers l'URL du backend

Important : Le placeholder `__PORT_8000__` doit être remplacé par l'URL réelle du backend (ex: `https://api.thesium.finance`).

7. Migration vers PostgreSQL (optionnel)

Le schéma SQLite utilisé par Thesium Desk est compatible avec PostgreSQL moyennant quelques ajustements mineurs. Cette migration est recommandée pour un déploiement en production multi-utilisateur.

Procédure de migration :

- Installer PostgreSQL et créer une base de données dédiée
- Installer le driver Python : `pip install psycopg2-binary` ou `pip install asyncpg`
- Modifier `DB_PATH` dans `models.py` pour utiliser la chaîne de connexion PostgreSQL
- Recréer les tables (le schéma est compatible avec des ajustements mineurs sur les types)
- Migrer les données existantes ou relancer `seed_data.py`
- Configurer la variable `DATABASE_URL` (voir section 5.2)

Avantages de PostgreSQL : support multi-worker, transactions concurrentes, meilleure performance en écriture, sauvegardes incrémentales.

8. Vérification Post-Installation

Après l'installation, utilisez la checklist suivante pour vérifier que l'ensemble des composants fonctionne correctement :

Test	Commande	Résultat attendu
Santé API	<code>curl /api/health</code>	<code>{"status": "healthy"}</code>
Dashboard	<code>curl /api/dashboard</code>	JSON avec portfolio, positions
Thèses	<code>curl /api/theses</code>	Liste de thèses actives
Instruments	<code>curl /api/instruments</code>	14 instruments
Cycle décision	<code>curl -X POST /api/orders/execute-cycle</code>	<code>{"success": true}</code>
Frontend	Ouvrir <code>index.html</code>	4 onglets, données affichées

Procédure de vérification complète :

```
# 1. Vérifier la santé de l'API
curl http://localhost:8000/api/health

# 2. Vérifier le dashboard
curl http://localhost:8000/api/dashboard | python -m json.tool

# 3. Vérifier les instruments
curl http://localhost:8000/api/instruments | python -m json.tool

# 4. Vérifier les thèses actives
curl http://localhost:8000/api/theses | python -m json.tool

# 5. Exécuter un cycle de décision
curl -X POST http://localhost:8000/api/orders/execute-cycle

# 6. Vérifier les mémos IC
curl http://localhost:8000/api/memos | python -m json.tool
```

Si tous les tests retournent les résultats attendus, l'installation est terminée avec succès.

9. Dépannage

Cette section répertorie les problèmes les plus fréquemment rencontrés et leurs solutions :

Problème	Cause possible	Solution
ModuleNotFoundError: fastapi	Dépendances non installées	<code>pip install -r requirements.txt</code>
Port 8000 déjà utilisé	Autre service sur ce port	Changer le port ou arrêter le service
Base de données vide	<code>seed_data.py</code> non exécuté	<code>python seed_data.py</code>
Erreur CORS frontend	Frontend sur un domaine différent	CORS configuré <code>allow_origins=["*"]</code>
Données de prix obsolètes	Pas d'ingestion récente	<code>POST /api/run-ingestion</code>
thesium.db verrouillé	Accès concurrent	Redémarrer le serveur

Commandes utiles de diagnostic :

```
# Vérifier la version de Python
python --version

# Vérifier que les dépendances sont installées
pip list | grep -E "fastapi|uvicorn"

# Vérifier que le port 8000 est libre
lsof -i :8000 # Linux/macOS
netstat -an | find "8000" # Windows

# Vérifier la taille de la base de données
ls -lh thesium.db

# Relancer le seed (réinitialise toutes les données)
rm thesium.db && python seed_data.py

# Lancer le serveur en mode debug
uvicorn api_server:app --host 0.0.0.0 --port 8000 --reload
```

Pour toute question supplémentaire, consultez le fichier `RUNBOOK.md` inclus dans le projet ou contactez l'équipe technique [Thesium.finance](#).

